

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Computer Graphics

Progress Report

Assigment 1

Advisor(s):	Trần Thị Ngọc Trâm	
Student(s)	Lê Công Minh Khang	2252295
	Philippe Belda	2660010

HO CHI MINH CITY, 06 OCTOBER 2026

Table of Contents

I.	Software structure	3
I.1.	Objects	4
I.2.	Scenes	4
II.	Shape Generators	4
II.1.	Data format	4
II.2.	Obj files	5
II.3.	2D shapes	5
II.3.1.	<code>__circularish</code> helper function	5
II.3.2.	Simple 2D shapes (triangle, rectangle, trapezoid, arrow)	6
II.3.3.	n-gon shapes (pentagon, hexagon, circle, ellipse)	6
II.4.	3D shapes	6
II.4.1.	<code>triangulationNation</code> helper function	6
II.4.2.	Cube	6
II.4.3.	Simple <code>__circularish</code> and <code>triangulationNation</code> shapes (cylinder, cone, truncated cone, tetrahedron, prism)	7
II.4.4.	Sphere	7
II.4.5.	Torus	8
II.4.6.	Surface	8
III.	Shaders	9
IV.	Lighting	9
V.	GUI	10
VI.	Challenges encountered	10

I. Software structure

The software has been structured to ensure modularity and concern separation. A first module is tasked to get the scene informations and pass it to the window manager. This modules handles the creation of the GL context the window management (display and hud). The draw calls at each frame are managed by the pipeline module that stores scene information as well as the wiring with GPU.

In Figure 1 is provided a more detailed diagram to better understand the sequence of operations and the responsibilities of each module.

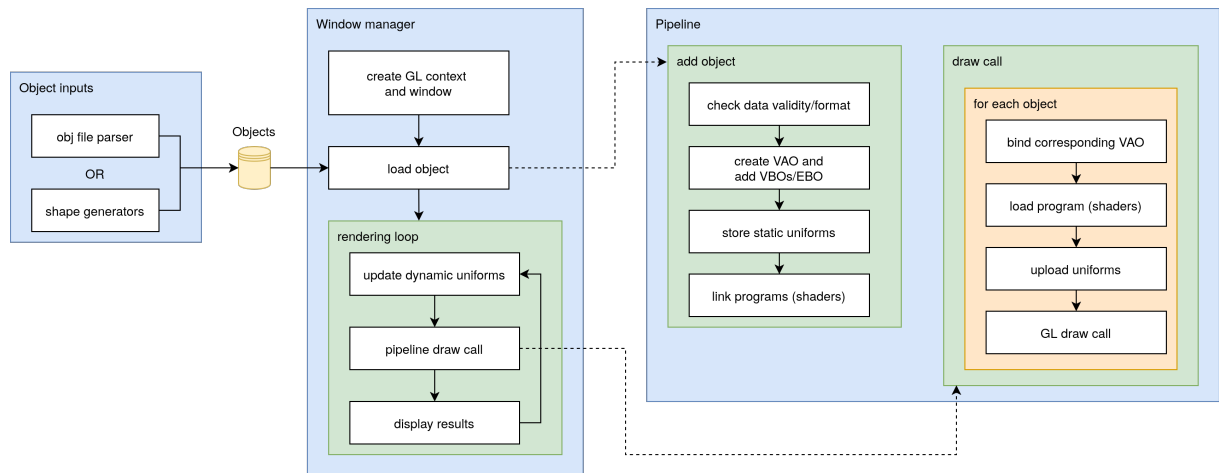


Figure 1: Diagram of the software structure

Our application initializes with an empty scene, with no objects displayed. The user can then select one of the registered scenes. To allow better performances and a fluid user experience, objects from the different scenes are cached. This results in short waiting time when changing scenes. The process is detailed in Figure 2.

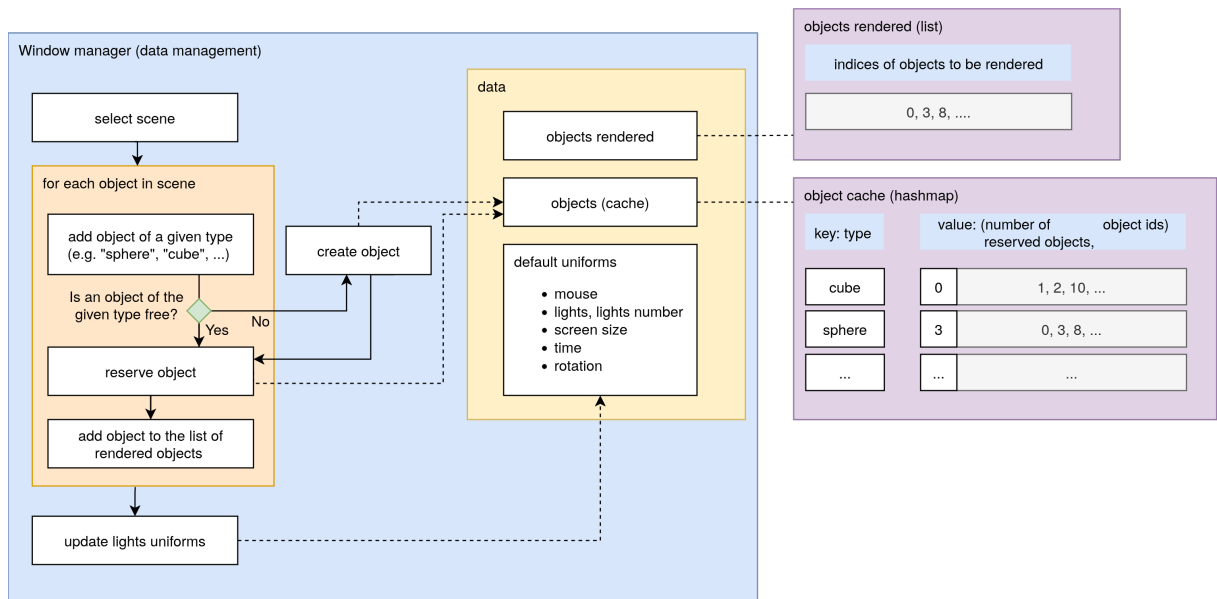


Figure 2: Details of the scene selection

I.1. Objects

In this project, objects are defined by their attributes:

- mesh: a mesh composed of vertices, normals and colors, stored in a VAO (Vertex Array Object)
- program: the program used to render the object. A single program can be used to render several objects.
- uniforms: as the programs are shared between objects, an objects can redefine some uniforms of the program to adapt the rendering

I.2. Scenes

Scenes are a collection of objects. The objects positions in the world are changed thanks to the `model` matrix uniform.

II. Shape Generators

II.1. Data format

We chose to bundle object data for rendering under the form:

```
data = {
    "vertices" # np.array((n, 3))
    "indices" # (optional): np.array((n)) if omitted, will be picking k-tuples
               from the vertices array where k is the size of primitive (eg 3 for triangles)
    "colors"   # np.array((n, 3)) each component must be in [0, 1]
    "normals": # np.array((n, 3))
    "mode":    # GL mode, indicating the rendering mode (eg: GL_TRIANGLES,
               GL_TRIANGLE_STRIP, ...)
}
```

All shape generators and `.obj` files parsing outputs are converted to this format. At the moment, for the sake of simplicity, the rendering mode is always `GL_TRIANGLES`, and colors and normals are automatically populated.

II.2. Obj files

From the wiki, we can see that `.obj` files are text files with the syntax:

```
<fieldtype> <field1> <field2> <etc..>
v 0.123 0.234 0.345 1.0
...
vt 0.500 1 [0]
...
vn 0.707 0.000 0.707
...
vp 0.310000 3.210000 2.100000
...
f 1 2 3
f 3/1 4/2 5/3
f 6/4/1 3/5/3 7/6/5
f 7//1 8//2 9//3
f ...
```

For now, we shall only focus on the 2 fields `v` (Vertex) and `f` (Triangle):

- For `data["vertices"]`, we parse the “v” fields as is
- For `data["indices"]`, the “f” fields in `.obj` files are 1-indexed when referencing which vertex is part of a triangle. As such, converting “f” fields to “indices” will need to decrement all referenced indices by 1. We collect only the X number in each pattern X/Y/Z.

II.3. 2D shapes

II.3.1. `__circularish` helper function

Generating circles comes up a few times in some of the 2D and 3D shapes generation methods, thus a helper function like `__circularish` is very convenient.

The exact functionality and implementation varies depending on use cases, which is why the function is copied accross files instead of declared and imported. In general, it takes in:

- `n`: number of vertices generated
- `size`: the distance the vertices will be from (0, 0, z)
- `z`: the z coordinate of vertices

It follows:

Given step size in angle:

$$\theta = \frac{2\pi}{n}$$

for `i` in `range(n)` we create a new vertex at coordinate:

$$(\cos(i\theta) \times \text{size}, \sin(i\theta) \times \text{size}, z)$$

II.3.2. Simple 2D shapes (triangle, rectangle, trapezoid, arrow)

These shapes are relatively simple, so each generator only takes as parameters a few basic arguments to scale them in different aspects. The vertices then gets their coordinate multiplied accordingly and populating `data["indices"]` is hardcoded.

II.3.3. n-gon shapes (pentagon, hexagon, circle, ellipse)

All of these shapes can be generated with some variation of the `__circularish` function. Notably:

- Pentagon: $n = 5$
- Hexagon: $n = 6$
- Circle: $n = 60$
- Ellipse: $n = 60$, with a multiplier to the width of the n-gon in one of the axis

These rings of vertices are triangulated to all have 1 common vertex, and then iterate around the n-gon for the other 2 vertices. This method is simple and ensures the correct ordering of the indices but can leads to rendering artifacts (pinching) around the common vertex. A better way would be to do it in zigzag strips, or manually implementing the `GL_TRIANGLE_FAN` mode (recall that we assume that all shapes will always render in `GL_TRIANGLES` mode).

II.4. 3D shapes

II.4.1. triangulationNation helper function

A lot of 3d shapes are more conveniently generated as quads (4 sided polygon). However, to be able to render in mode `GL_TRIANGLES`, we need to convert these lists of quads into a list of triangles.

It follows:

```
#      0.-----3
#      |\    |
#      | \   |
#      |  \  |
#      |   \ |
#      |    \|
#      1'-----'2
# inputQuad = [0,1,2,3]
res = []
for quad in listOfQuads:
    res.append([[quad[0],quad[1],quad[2]]])
    res.append([[quad[0],quad[2],quad[3]]])
return res
```

Note that the method requires that quads part of the input list needs to have vertices indexed in a counter clockwise direction. Usage of this method would need some consideration in how the `listOfQuads` are generated.

II.4.2. Cube

The simplest 3d shape, establishes the generation pattern that is used futher down the line.

```

p = size/2
obj["v"].append([ -p ,  p ,  p])
obj["v"].append([ -p ,  p , -p])
obj["v"].append([ -p , -p ,  p])
obj["v"].append([ -p , -p , -p])
obj["v"].append([  p , -p ,  p])
obj["v"].append([  p , -p , -p])
obj["v"].append([  p ,  p ,  p])
obj["v"].append([  p ,  p , -p])

listOfQuads = []
# top
listOfQuads.append([0,2,4,6])
# bottom
listOfQuads.append([7,5,3,1])

for i in range(0,8,2):
    listOfQuads.append([(i+0)%8,(i+1)%8,(i+3)%8,(i+2)%8])

```

We initialize vertices in pairs that creates 4 parallel edges of the cube going counter clockwise (in the code shown above it is parallel to the z axis).

Then quads of the top and bottom faces of the cube are hard coded, paying attention to “reverse” the order for the bottom face to ensure that it faces the correct way (backface culling).

Finally, as the order of vertex declaration is suitable, we can use a loop to iterate over the 4 side faces of the cube, with modulo operation % to be able to connect the last vertex pair to the first.

II.4.3. Simple `__circularish` and `triangulationNation` shapes (cylinder, cone, truncated cone, tetrahedron, prism)

All of these shapes are generated in a process similar to the Cube, as some combination of `__circularish` for the top/bottom faces and a loop to generate quads as side walls:

- Cylinder: 2 n-gon top/bottom faces with a list of quads connecting the two
- Prism: 2 triangle top/bottom faces with a list of quads connecting the two
- Truncated cone: a Cylinder with 1 of the circles scaled down
- Cone: an n-gon pyramid, with an n-gon bottom face and a triangle fan as the side connecting to a vertex at the top
- tetrahedron: a specifically scaled cone with a triangle as a base

II.4.4. Sphere

There are multiple ways to create a sphere, we chose uv sphere due to the similarity in construction with the earlier shapes.

In which, a sphere contains multiple `__circularish` n-gons of various sizes and z offsets, akin to the latitudes of a globe. These rings are connected to adjacent loops akin to how the sides of a cylinder was generated, only for multiple levels. At the pole, the nearest n-gon to the pole vertex is connected similar to how the cone was generated.

II.4.5. Torus

To generate a Torus, we need to take into consideration its properties. For its roundness, we can define it as cutting the torus in half (bisecting) through the plane containing the z axis (the center of the torus's hole) and have its cross section be a circle. The cross section of the torus should be a circle regardless of where you break it in half. In essence we are revolving a circle around some center axis (eg. the z axis in a 3d x y z coordinate system).

Because we are working with computers, it makes more sense to only generate a few slices, a few angles of the torus bisection to create a circular cross section, to avoid having to generate and render an infinite amount of vertices. So each "circular cross section" would be an n-gon shape and around the torus, there should only be m cross sections where m is a positive integer.

It is difficult to generate an n-gon cross-sections at different angles of bisection. So, instead of generating m n-gon cross sections, we can generate 1 piece of each m cross sections n times. To motivate this approach, imagine cutting the doughnut along the xy plane. You will see that the cross section now will be composed of 2 circles centered on the center of the torus' hole, one for the outside edge of the doughnut, one for the inner edge. We can think of making m pieces with an `__circularish` generator producing m-gons that are centered on the axis of revolution (the z axis). These m-gons would have its radius and z offset borrowed from a point along the surface of the torus.

This means, in practice:

- we first generate a temporary n-gon offset from the z axis
- borrow the measurements of its vertices to generate our m-gons making up the structure of our torus
- have some clever loops to connect these m-gons' vertices together to form the surface of the torus

II.4.6. Surface

Surface rendering takes in a function $z = f(x,y)$ where $f(x,y)$ is a python lambda: example:

```
func = lambda x, y: numpy.sin(x) + numpy.cos(y)
```

It also takes in parameters to control the limits of the surface to be shown in the x and y direction (`+limx` and `+limy`).

We consider a small slice of the xyz coordinate space where we want to render the surface of our function. It spans `+limx` in the x direction, `+limy` in the y direction and infinitely in the z direction. We populate it evenly with a vertex every dx and dy, and have that vertex's z offset be determined by the lambda function provided. Lastly, we can connect these vertices with their neighbors to create the surface of our function. Note that this implementation can not handle illegal (eg. divide by 0) or limits and can't render (or render accurately) discontinuous functions.

III. Shaders

In a given scene, each object is associated with a shader program. A shader program is created from 2 shader: a vertex shader and a fragment shader. Specifically, when a program is created (in `Pipeline.update_program`), the 2 shader files are uploaded to the GPU before being compiled. They are then linked into a program, and the 2 former shader objects are destroyed.

However, the shaders might need some associated data: uniforms and textures. The `UManager` class is charged to handle these.

- uniforms:

The uniforms are separated in 2 categories: static and dynamic.

As the name suggests, the static ones stay the same during the whole execution of the program, while the dynamic ones change from frame to frame.

There are currently only 2 dynamic uniforms: one to track the time passed from the start of the execution (`float iTime`), and another to track the mouse position (`vec4 iMouse`). Dynamic uniforms are computed for each frame and are declared in `windows.py`, thus this type of uniforms is not available for the final user. However, static uniforms can be defined by the user and passed as parameters in `Pipeline.add_object` during the creation of each object.

- textures:

Currently textures are not supported.

IV. Lighting

The lighting of a scene can completely change its final rendering, hence it plays a major role in computer graphics. In this project, we chose to handle scene lights thanks to a data structure that is passed to the fragment shader as a uniform. This uniform is in fact an array of lights, to enable the definition of multiple lights. A light is declared in python as an instance of the class `Light`, that is simply a container to hold the attributes defining the light.

Currently, the class is as follow:

```
class Light:
    def __init__(
        self,
        position,
        color=(1, 1, 1),
        intensity=1.0,
        # dtype=1,
        # constant=1.0,
        # linear=0.09,
        # quadratic=0.032,
    ):
        self.position = position
        self.color = color
        self.intensity = intensity
        # self.dtype = dtype
        # self.constant = constant
```

```
# self.linear = linear
# self.quadratic = quadratic
```

With the light passed down to the fragment shader, we simply have to apply a formula (Goureaud, Phong, ...) to obtain the fragment color.

V. GUI

The graphical user interface (GUI) is composed of a GL display window left and an option panel right.

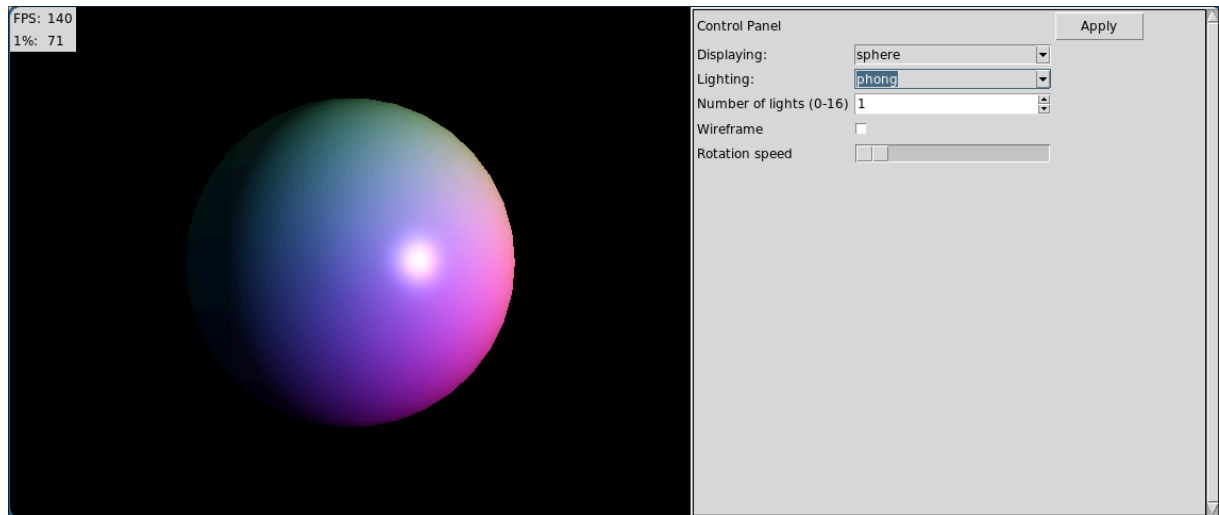


Figure 3: Application interface

The left panel displays the scene selected by the user on the left panel under the option “Displaying”. In the top left corner a little window indicates the frame displayed per second as well as the frame rate lowest 1% computed on a 1s time window. This panel supports mouse interaction through the use of camera: the user can change the direction of the camera by dragging the mouse with left click, zoom with mouse wheel and change the camera position in the scene by dragging the mouse with right click.

The right panel provides the ability for the user to change the scene itself. They thus can select a predefined scene or import an .obj file under the “Scene” section. The “Lighting” option apply a given shading program to every object in the scene (some scenes may have objects with different programs). Other options allow the user to change the number of lights, enable wireframe mode or change the rotation speed of the scene.

VI. Challenges encountered

The integration of cameras inside the project architecture raised some questions about the project modularity and organisation. Indeed, cameras are needed to represent a 3D scene and one could decide to not provide any support from the library and ask the user to compute the view and projection matrices directly inside the vertex shader. This raises concerns about inputs, that then have to pass through uniforms, and about debugging. Indeed, debugging a Python code is much more convenient than debugging one written in GLSL, especially when it comes to matrix computations and hardware compatibility issues.

This reasons led us to create the **Camera** class, a pattern providing 3 main functions: `view_matrix`, `_projection_matrix` and `update`. The first 2 return the corresponding matrices and the latter returns an array of uniforms updated with the “projection” and “view” fields.

Then, once a camera implementation is passed to the main program, the user simply has to collect the “view” and “projection” uniforms in the vertex shader to compute the final vertices position.

List of Figures

Figure 1 Diagram of the software structure 3

Figure 2 Details of the scene selection 4

Figure 3 Application interface 10